

named entity recognition. In *Proceedings of the Seventh Conference on Natural Language Learning at HLT-NAACL 2003*, pages 142–147, 2003.

- [51] M. Wang, H. Chau, K. Thaker, P. Brusilovsky, and D. He. Knowledge annotation for intelligent textbooks. *Technology, Knowledge and Learning*, 28(1):1–22, Mar 2023.
- [52] X. Yang, J. Bian, W. R. Hogan, and Y. Wu. Clinical concept extraction using transformers. *Journal of the American Medical Informatics Association*, 27(12):1935–1942, December 2020.
- [53] Z. Yang, Z. Dai, Y. Yang, J. Carbonell, R. Salakhutdinov, and Q. V. Le. XLNet: Generalized autoregressive pretraining for language understanding. In *Proceedings of the 33rd Conference on Neural Information Processing Systems (NeurIPS)*, Vancouver, Canada, December 2019.
- [54] J. Zhu. Text mining and topic modelling in english teaching: Extracting key themes and concepts for effective curriculum development. *Journal of Computational Methods in Sciences and Engineering*, 25(2):1210–1222, 2025.

Acknowledgments

For anonymity, the acknowledgments are left blank.

APPENDIX

A. CODEBOOK

A.1 Purposes

The purpose of this codebook is to provide a standard for manually annotating concepts in a course lecture presentation for the purposes of creating a dataset of annotated concepts. This dataset may be used to train a model to automatically extract concepts from course material. The codebook is partially based on insights from [51].

To perform the annotation, we use a custom annotation tool where concepts are selected by clicking them. This allows us to identify the concepts in the text by highlighting them, and we can then export the annotations into a format useful for training our model.

In this codebook, we define a concept as follows:

A word/series of words is a **concept** in a presentation if it represents a complete unit of knowledge taught in the lecture that the instructor wanted the students to take away.

A.2 Dataset Annotation Guidelines

Annotators will work on units of a single slide deck (lecture) at a time. A concept is labeled based on the following assumptions: Terms that are defined, explicitly or implicitly, in the lecture are marked as Concepts. Terms that are extensively focused on in the text of the lecture, will be marked as concepts. Only consider a term a concept if its justification exists within the slide deck itself. Do not infer concepts from outside knowledge, prior lectures, or general domain

assumptions. For our purposes, only consider computer science concepts. Mathematical and statistical concepts, if relevant, may also be considered concepts. Coders should read the slide deck in chronological order. Label all instances of a concept. On average, expect to spend one minute to label each slide.

Abbreviation Rule

If an abbreviation for a concept is in the text, also mark that as a concept. For example, see Interrupt Service Routines and ISRs in the following labeled text.

Interrupts are handled using **Interrupt Service Routines (ISRs)**

- **ISRs** are segments of code that determine what action should be taken for each type of **interrupt**

Concepts are typically nouns and not adjectives or verbs. However, there are notable exceptions, as mentioned in the next few guidelines.

Variant Form Rule

Mark variant forms of a concept as a concept, such as the plural form or adjectival form, except for possessives. For example, “tokens” and “token” are both labeled as concepts in the following, but “lexer” is labeled as a concept and not “lexer’s”:

the **lexer’s** output is a list of **tokens**

- each **token** is like one “word” in the **source language** – the smallest unit of meaning

Verb Form Subrule

We consider verb versions of concepts to be concepts as well. For example, if lexing were in the above text, we would label it as a concept.

Mathematical Symbol Rule

Sometimes a mathematical symbol will abbreviate a concept. Label these as concepts. For example, take this section from a Theory of Computation lecture, where Σ stands in for the alphabet.

$$\Sigma = \{0, 1, \#\}$$

Adjoinment Rule

Annotate concepts at the most specific level that adds meaningful information. If subdividing a phrase adds no additional meaning or context, e.g. “OS kernel”, treat it as a single concept.

part of the **OS kernel**

On the other hand, if the smaller components each carry distinct meaning, annotate them separately.

the **OS** interrupts the program

Generic Term Rule

Avoid designating phrases as concepts that are generic, like machine or copy. Only if the course assigns those terms course-specific meaning should they be designated as concepts. For example we would not label machine or copy here:

The machine splits into multiple copies of itself.

But we would label language and recognize here:

The **Turing Machine** recognizes the language **L**

Generic Alternatives Rule

If a concept can also be called by an alternative generic term, do not label that alternate term UNLESS that term is used extensively within the lecture to refer to the concept. For example, if “text” is provided as an alternative for code but the lecture does not use it to refer to code in subsequent slides, we do not label it as a concept.

There are two main parts of a program: **code** and **data**.

- “**code**” is sometimes called “text”

But here, both 1-1 function and injective function are used extensively:

A **one-to-one function** (also called an **injective function**) is a **function** f such that $f(x) = f(y) \implies x = y$

Umbrella Polysemy Rule

If a general word is used as an umbrella term to refer to multiple concepts, do not label it as a concept. In the example that follows, “kernel mode” and “user mode” are concepts, but “modes” is not.

A processor can be in **kernel mode** or **user mode**. These modes help provide **OS** security.

Sub-Concept Elevation Rule

If a concept has a sub-concept, and that sub-concept is used multiple times, then label it as a concept, demonstrated below.

The **classification task** is to sort [Note: sort here is a generic term] data into categories. **Binary classification** is **classification** but restricted to two categories.

Named Example Rule

If a named example is given in a text, label it only as a concept if it is covered in detail. For example, if an OS course mentioned the Linux File System in reference to file systems, and then spent a significant portion of the lecture exploring it, then label it as a concept, as below:

File systems are a key component of the **operating system**. They determine how **files** are structured and **indexed** on the **file system**. One notable example of this is the **Linux File System**, which we shall study in further detail later on in the lecture.

But if, for example, Linux were given as an example of an operating system and the lecture did not go into any details, then do not label it as a concept.

Operating systems are programs that act as an intermediary between a user of a computer and the computer hardware. Some examples include Windows, MacOS, FreeBSD, Linux, and Unix.

Equation Labeling Rule

The only things to label in mathematical equations are the functions that stand in for concepts from outside of the equations, like the greatest common denominator function, gcd, or Euler’s totient function, φ . See the following:

What the φ ?

Here, we mean φ to be **Euler’s totient**

$$\varphi(p) = p - 1.$$

Code Labeling Rule

Modifiers, keywords, or syntactic elements are **not** labeled unless the slide deck explicitly defines or discusses them as concepts (e.g., **static**, **visibility**).

```
while(){...}
```

- **While** a **condition** is **true**, run the block
- **While** not at the end of the list, print out the next element

```
while(not at end of list)
{
    System.out.println(next element);
} //end loop while(not at end of list)
```

Position Agnosticism Rule

Ignore the position of the text, even if a concept is in the header or footer of a slide, label it as a concept.

Out of Discipline Rule

If a term is not within the area of computer science, but is within mathematics or statistics, still label it as a concept,

like "least squares regression" in the following. If it is a concept outside of the scope of the course, like years of potential life lost in the following, do not mark it as a concept.

Least squares regression is a statistical technique for determining a **line of best fit**. For example, determining the years of potential life lost. A measure of the impact of premature mortality on a population.

Conjoined Separation Rule

Consider the phrase "arithmetic and geometric sequences." Even though "arithmetic sequences" is a concept, we cannot annotate it using our system. Thus, for simplicity's sake, we should only label geometric sequences as a concept, as seen below.

...arithmetic and **geometric sequences**.

Typo Rule

If a concept is present, but has a typo, do not correct the typo and do not annotate the concept, as seen with the misspelling of interrupts as interups below:

...interups play a big role in **operating systems**.

B. GPT 5 PROMPT

The following is the full prompt provided to GPT 5, excluding the text of the lecture.

ROLE

You are an expert academic annotator specializing in **concept extraction** from university lecture slides.

Your goal is to identify and list only the key ***concepts*** that are **explicitly** defined, emphasized, or used in examples within the provided slide text.

Do NOT infer concepts beyond the slide text. Your task is purely textual and evidence-based.

CODEBOOK (Condensed from ■Concept Annotation Codebook - Refined, Oct 2025■)

1. ***Slide-Deck Grounding Rule*** -- Only annotate concepts that are justified or defined in the given slide text. Do not infer from outside knowledge.

2. ***Granularity Rule*** -- Annotate at the most specific level that adds distinct meaning.

- If splitting a phrase adds no new meaning, keep it as one concept.
- If the parts are meaningful alone, include both the whole and its parts.

3. ***Definition \& Emphasis Rule*** -- Label any word or phrase that is:

- Clearly **defined or described**,
- **Emphasized** as a key term, or
- Used as the **subject** of an example or explanation.

4. ***Abbreviation Rule*** -- Always include abbreviations

and their expanded forms on separate lines.

5. ***Non-Conceptual Modifiers Rule*** -- Do ***not*** include syntax, keywords, or modifiers unless the slide explicitly defines or discusses them as concepts (e.g., ■**static**■, ■**visibility**■).

6. ***Adjoinment Rule*** -- Include multi-word terms when they represent a unified idea (e.g., ■**function header**■, ■**process control block**■).

7. ***Generic Term Rule*** -- Do not label generic or context-free terms (e.g., ■**thing**■, ■**object**■) unless defined specifically.

FILTERING STEP

After identifying possible concepts, remove any that are ***not explicitly***:

- defined,
- emphasized, or
- used in an example or explanation.

Only keep the **most important instructional concepts**.

OUTPUT REQUIREMENTS

- Output ***each concept on a new line***.
- Do ***not*** number them.
- Do ***not*** include reasoning, commentary, or explanations.
- Include abbreviations **and** their full forms as separate lines.
- Keep consistent capitalization (use lowercase unless the concept is an acronym).
- Do not include REPEATED COURSE NAME as a concept.

Example Output:

function
function name
function body
function header
parameters
return type

EXAMPLE ANNOTATION:

(SLIDE TEXT)

[SLIDE TEXT FOR INTRO TO PROGRAMMING LECTURE 7 HERE]

(ENDING CONCEPTS)

Function
Functions
Parameter
Parameters
function body
return
Returns
function header
function name
header
return type
scope
passing-in
pass-by-value
pass-by-reference
void
variable
variables